

Removing Interference of Conditions for Disjoint Parallel Programs (continued)

Let us look at one more example of removing interference in conditions for a disjoint parallel program.

1. Let's look at Example 4 in Lecture 23 again. Create a full proof outline for $\{x = y = z = c\} [x := x^2 \parallel y := y^2 \parallel z := (z - d) * (z + d)] \{x = y = z + d^2\}$, where c, d are named constants.

- a) Without the provided precondition in each thread, it is very natural to use $x = y = z = c$ as the precondition of every thread, then we have the following **wrong** partial proof outline if we use the parallelism rule:

$$\begin{aligned} & \{x = y = z = c\} \\ & [\{x = y = z = c\} x := x^2 \{x = c^2 \wedge y = z = c\} \\ & \parallel \{x = y = z = c\} y := y^2 \{y = c^2 \wedge x = z = c\} \\ & \parallel \{x = y = z = c\} z := (z - d) * (z + d) \{z = c^2 - d^2 \wedge x = y = c\}] \\ & \{x = c^2 \wedge y = z = c \wedge y = c^2 \wedge x = z = c \wedge z = c^2 - d^2 \wedge x = y = c\} \end{aligned}$$

It is wrong since interference in conditions happens everywhere. We need to age all the variables in preconditions.

- b) For the first thread: $\{x_0 = y_0 = z_0 = c \wedge x = x_0\} x := x^2 \{x_0 = y_0 = z_0 = c \wedge x = x_0^2\}$.
- c) For the second thread: $\{x_0 = y_0 = z_0 = c \wedge y = y_0\} y := y^2 \{x_0 = y_0 = z_0 = c \wedge y = y_0^2\}$.
- d) For the third thread: $\{x_0 = y_0 = z_0 = c \wedge z = z_0\} z := (z - d) * (z + d) \{x_0 = y_0 = z_0 = c \wedge z = z_0^2 - d^2\}$.
- e) In the end, we get the following full proof outline:

$$\begin{aligned} & \{x_0 = y_0 = z_0 = c \wedge x = x_0 \wedge y = y_0 \wedge z = z_0\} \\ & [\{x_0 = y_0 = z_0 = c \wedge x = x_0\} x := x^2 \{x_0 = y_0 = z_0 = c \wedge x = x_0^2\} \\ & \parallel \{x_0 = y_0 = z_0 = c \wedge y = y_0\} y := y^2 \{x_0 = y_0 = z_0 = c \wedge y = y_0^2\} \\ & \parallel \{x_0 = y_0 = z_0 = c \wedge z = z_0\} z := (z - d) * (z + d) \{x_0 = y_0 = z_0 = c \wedge z = z_0^2 - d^2\}] \\ & \{x_0 = y_0 = z_0 = c \wedge x = x_0^2 \wedge y = y_0^2 \wedge z = z_0^2 - d^2\} \\ & \{x = y = z + d^2\} \end{aligned}$$

Observations about the Parallelism Rule and Interference Freedom

- So far, our requirements for using the Parallelism Rule is that the parallel program must be a disjoint parallel program, and it has disjoint conditions. Is this requirement too strict? Yes! What we need for applying the parallelism rule is that “threads should not interfere with each other” or “interference free”. Let's look at some examples and think about what is actually needed for “interference free”.
2. Let's look at the following threads with their provable pre- and post- conditions, can we use parallelism rule to prove the parallel composition of them?

- a) $\{x = 0\} y := y + 1 \{x = 0\}$
 $\{y > 1\} x := 0 \{y > 1\}$

Yes, $\{x = 0 \wedge y > 1\} [y := y + 1 \parallel x := 0] \{x = 0 \wedge y > 1\}$ is **totally correct**; even though these **disjoint threads don't have disjoint conditions**.

b) $\{x = 0\} y := y + 1 \{x = 0\}$
 $\{z = 0\} y := y + 2 \{z = 0\}$

Yes, $\{x = 0 \wedge z = 0\} [y := y + 1 \parallel y := y + 2] \{x = 0 \wedge z = 0\}$ is **totally correct** even though these **two threads are not disjoint**.

c) $\{x > 0\} x := x + 1 \{x > 1\}$
 $\{x > 0\} x := x + 2 \{x > 2\}$

Yes, $\{x > 0\} [x := x + 1 \parallel x := x + 2] \{x > 1 \wedge x > 2\}$ is **totally correct**; even though **these two threads are not disjoint, and they don't have disjoint conditions**.

- So that we can use the parallelism rule to prove more parallel statements, we want to find out what makes a thread not interfere the execution of another thread.

3. Let's look at the two following pairs of threads.

a) We can apply the parallelism rule on these two threads:

$$\frac{\{x > 0\} x := x + 1 \{x > 1\}}{\{x > 0\} x := x + 2 \{x > 2\}}$$

b) We cannot do the same for these two threads.

$$\frac{\{x = 0\} x := x + 1 \{x = 1\}}{\{x = 0\} x := x + 2 \{x = 2\}}$$

What is the difference?

In a), given the precondition of the second thread $x > 0$ being true, the execution of $x := x + 1$ in the first thread doesn't change the correctness of $x > 0$; similarly, given the postcondition of the second thread $x > 2$ being true, the execution of $x := x + 1$ in the first thread doesn't change the correctness of $x > 2$.

Also in a), given the precondition of the first thread $x > 0$ being true, the execution of $x := x + 2$ in the second thread doesn't change the correctness of $x > 0$; similarly, given the postcondition of the first thread $x > 1$ being true, the execution of $x := x + 2$ in the second thread doesn't change the correctness of $x > 1$.

The same statements don't hold for the threads in b).

- From the above observations, we define the **interference-freedom check** for **atomic statement** $\{p\} S \{...\}$ versus a predicate q is the triple $\{p \wedge q\} S \{q\}$: if this triple is totally valid, then we say that $\{p\} S \{...\}$ **does not interfere with** q .
 - Similar to the "disjointedness test" and "disjoint condition test", this interference-freedom check is also static and can be too strict.

4. Consider the following pairs of statement $\{p\} S \{...\}$ and predicate q , does the statement interfere with q ?

a) $\{p\} x := x + 1 \{...\}$ and $x \geq 0$.

No. The triple we need to check its validity is $\{p \wedge x \geq 0\} x := x + 1 \{x \geq 0\}$, and it is **totally valid**.

b) $\{p\} x := x + 1 \{...\}$ and $x < 0$.

Yes. The triple we need to check its validity is $\{p \wedge x < 0\} x := x + 1 \{x < 0\}$, it is **not valid** since state $\{x = -1\}$ doesn't satisfy it.

- c) $\{p\} S \{...\}$ and F .

No. The triple we need to check its validity is $\{p \wedge F\} S \{F\}$, and it is **totally valid**.

- d) $\{x \% 4 = 2\} x := x + 4 \{...\}$ and $x \% 2 = 0$.

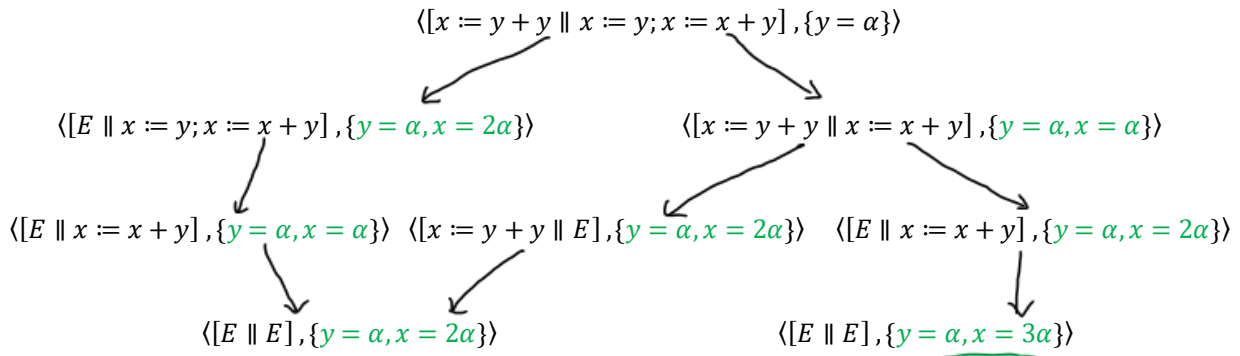
No. The triple we need to check its validity is $\{x \% 4 = 2 \wedge x \% 2 = 0\} x := x + 4 \{x \% 2 = 0\}$, and it is **totally valid**.

- For threads $\{p_1\} S_1 \{q_1\}$ and $\{p_2\} S_2 \{q_2\}$, what are the interference-freedom check we need to decide their interference freedom?
 - Someone may say the four following checks are enough:
 - $\{p_1\} S_1 \{...\}$ vs p_2 , thus the triple $\{p_1 \wedge p_2\} S_1 \{p_2\}$
 - $\{p_1\} S_1 \{...\}$ vs q_2 , thus the triple $\{p_1 \wedge q_2\} S_1 \{q_2\}$
 - $\{p_2\} S_2 \{...\}$ vs p_1 , thus the triple $\{p_2 \wedge p_1\} S_2 \{p_1\}$
 - $\{p_2\} S_2 \{...\}$ vs q_1 , thus the triple $\{p_2 \wedge q_1\} S_2 \{q_1\}$

They are **probably not enough**. The problem is that: what if S_1 is a “long statement” (not atomic) and we execute S_1 for several steps then switch to S_2 then back to S_1 ? Do we need to consider other pre- and post- conditions in the proof outline of S_1 ? Let’s look at the following example.

5. Let $S \equiv [x := y + y \parallel x := y; x := x + y]$ and we execute S in a state $\{y = \alpha\}$ (which satisfies the precondition of the program). The postcondition we want for this program $x = 2\alpha$. Are there any race conditions in this program?

- We can get the following evaluation graph:



- We can see that $M(S, \{y = \alpha\}) = \{\{y = \alpha, x = 2\alpha\}, \{y = \alpha, x = 3\alpha\}\}$, and $\{y = \alpha, x = 3\alpha\}$ doesn’t satisfy the postcondition we need. However, if we look at these two threads separately, we see that $x := y + y$ and $x := y; x := x + y$ can both give us $\{y = \alpha, x = 2\alpha\}$.
- The invalidity here is caused by **interleaving** a thread: we execute one thread for some steps then switch to another thread before finishing the execution. The execution orders of threads causes this invalidity, so the program has race conditions.
- For different types of statement S as a thread, when can “interleaving thread S ” occur?
 - An assignment statement or a **skip** statement is atomic itself; we cannot interleave it.
 - Interleaving a sequential statement can occur after the execution of one statement and before another one in the sequential.

- For **if – fi** and **while** statements, interleaving the execution of loop body or the execution of a branch can occur and can be harmful. Also, although evaluation of a Boolean expression is atomic, interleaving can occur between inspection of the result and the jump to the next configuration.
 - For example, if $\sigma(x) = 0$, then $\langle \text{if } x = 0 \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma \rangle \rightarrow \langle S_1, \sigma \rangle$. In parallel, another statement can be executed between the **if** condition test and the start of the true branch. For example, it is possible that we have $\langle [\text{if } x = 0 \text{ then } S_1 \text{ else } S_2 \text{ fi} \parallel x := x + 1], \sigma \rangle \rightarrow \langle [S_1 \parallel x := x + 1], \sigma \rangle \rightarrow \langle [S_1 \parallel E], \sigma[x \mapsto 1] \rangle$. In this last configuration, we're about to execute S_1 not in σ , but in $\sigma[x \mapsto 1]$. This is a problem because if x is now evaluated to 1 and the condition $x = 0$ is not true anymore and we supposed to execute S_2 instead.
- Are all the interleaving harmful? Let's look at this example.
- 6. Does the program $\{x \geq 0\} [x := x + 1; x := x + 2 \parallel x := x + 3] \{x \geq 6\}$ have race conditions?
 - **No**, no matter whether we interleave the first thread or not, we always increase x 's value by 6 after the execution.
- The above example shows that not all the interleaving is harmful. Informally, we define a part of the thread that must be executed atomically (no interleaving) to get the expected postcondition as a **critical section**. The race conditions caused by interleaving a critical section is called a **critical section problem**.
 - For example, in the program in Example 5, " $x := y; x := x + y$ " should be a critical section.
 - In the program in Example 6, there are no critical sections.
- To decide whether an interleaving is harmful needs us to analyze the semantic of the parallel program.
- To avoid critical section problems, we can clarify critical sections in threads by defining the **atomic regions**:
 - If S is a statement, then $\langle S \rangle$ is an **atomic region statement (or atomic statement in short)** with body S . The evaluation of S is considered as one step: If $\langle S, \sigma \rangle \rightarrow^k \langle E, \tau \rangle$ and $k \geq 1$ then $\langle \langle S \rangle, \sigma \rangle \rightarrow \langle E, \tau \rangle$.
- Using atomic regions gives us control of the occurrence of possible interleaving; however, making more or larger atomic sections is no panacea: The more or larger atomic regions code has, the less interleaving and hence parallelism we have.

Now, let's come back to interference-freedom checks for threads $\{p_1\} S_1 \{q_1\}$ and $\{p_2\} S_2 \{q_2\}$. Since we are considering possibilities of interleaving threads, we need to consider the proof outlines of these threads. Let's denote the (partial or full) proof outline of $\{p_i\} S_i \{q_i\}$ as $\{p_i\} S_i^* \{q_i\}$.

- The atomic statement $\{p_1\} T_1 \{...\}$ does not interfere with the proof outline $\{p_2\} S_2^* \{q_2\}$, if $\{p_1\} T_1 \{...\}$ doesn't interfere with p_2 nor with q_2 , nor with any precondition before an atomic statement in S_2^* (i.e. any r where $\{r\} < \dots > \{...\}$ appears in S_2^*).
- A proof outline $\{p_1\} S_1^* \{q_1\}$ does not interfere with the proof outline $\{p_2\} S_2^* \{q_2\}$, if every atomic statement $\{r\} T_1 \{...\}$ in S_1^* doesn't interfere with $\{p_2\} S_2^* \{q_2\}$.
- Two proof outlines $\{p_1\} S_1^* \{q_1\}$ and $\{p_2\} S_2^* \{q_2\}$ are **interference free** if they don't interfere with each other.

7. How do we decide whether statement $\{x > 0\} x := x + 1 \{x > 1\}$ interfere with the following thread (written in a partial proof outline)?

$S^* \equiv$

$\{x > 0\} y := 5; \{x > 0 \wedge y = 5\} z := 0; \{x > 0 \wedge y = 5 \wedge z = 0\}$

$\{\text{inv } x + z \geq 5\}$

$\langle \text{while } x > y \text{ do}$
 $\{x + z \geq 5 \wedge x > y\}$

$x := x - 1; z := z + 1 \{x + z \geq 5\}$
od $\{x + z \geq 5 \wedge x \leq y\}$

- We need to do the interference-freedom check of statement $\{x > 0\} x := x + 1 \{...\}$ vs the following predicates:
 - $x > 0$, thus check the validity of triple $\{x > 0\} x := x + 1 \{x > 0\}$ ✓
 - $x > 0 \wedge y = 5$, thus check the validity of triple $\{x > 0 \wedge y = 5\} x := x + 1 \{x > 0 \wedge y = 5\}$ ✓
 - $x + z \geq 5$, thus check the validity of triple $\{x > 0 \wedge x + z \geq 5\} x := x + 1 \{x + z \geq 5\}$ ✓
 - $x + z \geq 5 \wedge x \leq y$, thus check the validity of triple $\{x > 0 \wedge x + z \geq 5 \wedge x \leq y\} x := x + 1 \{x + z \geq 5 \wedge x \leq y\}$ ✗

These predicates are either the very first precondition, the very last postcondition of S^* , or they are the precondition of some atomic statement in S^* .

- Why don't we check whether a statement interferes with the postcondition of an atomic region or the pre- and post- condition of a non-atomic statement?
 - Interleaving is allowed after the execution of an atomic region and in non-atomic regions. As long as a statement does not interfere with the precondition of an atomic statement, we can switch back to this thread and execute the atomic region without problems.

8. Are the following threads interference free? Here predicate function $even(x) \equiv x \% 2 = 0$

$S_1^* \equiv \{even(x)\} x := x; \{even(x)\} x := x + 2 \{even(x)\}$

$S_2^* \equiv \{x > 2\} \text{ if } even(x) \text{ then}$
 $\{x > 2 \wedge even(x)\} x := x - 2 \{x > 0 \wedge even(x)\}$
 $\text{else } \{x > 2 \wedge \neg even(x)\} x := x - 1 \{x > 1 \wedge even(x)\} \text{ fi}$
 $\{x > 0\}$

- To decide whether S_2^* interferes with S_1^* , we need to check the validity of the following triples:
 - $\{x > 2 \wedge even(x)\} x := x - 2 \{even(x)\}$. It is totally valid.
 - $\{x > 2 \wedge \neg even(x) \wedge even(x)\} x := x - 1 \{even(x)\}$. It is totally valid.
- To decide whether S_1^* interferes with S_2^* , we need to check the validity of the following triples:
 - $\{even(x) \wedge x > 2\} x := x \{x > 2\}$. It is totally valid.
 - $\{even(x) \wedge x > 2 \wedge even(x)\} x := x + 2 \{x > 2 \wedge even(x)\}$. It is totally valid.
 - $\{even(x) \wedge x > 2 \wedge \neg even(x)\} x := x \{x > 2 \wedge \neg even(x)\}$. It is totally valid.
 - $\{even(x) \wedge x > 0\} x := x \{x > 0\}$. It is totally valid.
 - $\{even(x) \wedge x > 2\} x := x + 2 \{x > 2\}$. It is totally valid.
 - $\{even(x) \wedge x > 2 \wedge even(x)\} x := x + 2 \{x > 2 \wedge even(x)\}$. It is totally valid.
 - $\{even(x) \wedge x > 2 \wedge \neg even(x)\} x := x + 2 \{x > 2 \wedge \neg even(x)\}$. It is totally valid.
 - $\{even(x) \wedge x > 0\} x := x + 2 \{x > 0\}$. It is totally valid.
- Thus, $\{p_1\} S_1^* \{q_1\}$ and $\{p_2\} S_2^* \{q_2\}$ are interference free.

Now, we can update the Parallelism Rule:

- **Parallelism Rule**

For $k = 1 \dots n$, if proof outlines $\{p_k\} S_k^* \{q_k\}$ are interference free:

1 $\{p_1\} S_1^* \{q_1\}$

$$2 \{p_2\} S_2^* \{q_2\}$$

...

$$n \{p_n\} S_n^* \{q_n\}$$

$$n+1 \{p_1 \wedge p_2 \wedge \dots \wedge p_n\} [S_1^* \parallel S_2^* \parallel \dots \parallel S_n^*] \{q_1 \wedge q_2 \wedge \dots \wedge q_n\} \quad \text{parallelism } 1, 2, \dots, n$$